x.-app:

-observing app behaviour

-language:

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch:

2.-app:acala studio

-observing app behaviour

-language: c++

-tool: die,x64dbg

-protection: -

-identify: by string ref

-binary patch: method 1: jmp change

method 2: nop eax ret function that triger bad branch

3.-app: personality evluation program

-observing app behaviour

-language:vb6

-tool: die,vb8decompiler

-protection: -

-identify:

-binary patch: method 1:

-done watch: no

4.-app: hjzn trade copy

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

5.-app: pixtopian book

-observing app behaviour

-language: c++

-tool: die,x32dbg

-protection: -

-identify: call stack method(function dont have name),string find pattern in memory map

-binary patch: method 1: jmp change byte(found by xanalyzer auto comment)

-step analysis: identify second uper stack that coment from pixtobian.function_adress, go to it identify message box just by

number parameter although function name dont show real name(messagebox), because there is no branch in that function

good and bad branch is in previous function(callee function)

-some note: in code data that push data string can see with right click->follow in dump in that code

-done watch: yes

6.-app: visual site designer

-observing app behaviour

-language: c++

-tool: die,x32dbg

-protection: -

-identify:

-binary patch: method 1:

-some note: to close nag or skip it by by in call stack but with system party not user party

-done watch: yes

7.-app: pc surgeon

-observing app behaviour

-language: delphi

-tool: die,x32dbg

-protection: -

-identify: string ref

-binary patch: method 1:jmp change byte

-done watch: yes

8.-app: mr bill

-observing app behaviour

-language: c+=

-tool: die,

-protection: -

-identify: string ref

-binary patch: method 1: al change byte function that triger bad branch

-done watch: yes

9.-app: artgem

-observing app behaviour

-language: c++

-tool: die,x32dbg

-protection: -

-identify: string ref

-binary patch: method 1: nop eax in function return that triger bad branch

-done watch: yes

10.-app: jigsaw

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-identify: string ref

-binary patch: method 1:

-done watch: yes

11.-app: cyclone

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: string ref(folder path string)

-binary patch: method 1: nop branch

-done watch: yes

12.-app: power iso

-observing app behaviour

-language: c++

-tool: die, x32dbg, resource hacker(for remove start nag)

-protection: -

-identify: string ref, call stack function

-binary patch: method 1: jmp change byte

-some note: in call stack method, find function that have bad message but no branch to good message, mean that function

for bad function, good branch is caller function, if not in 1st caller then up to 2nd caller,so on. dont forger

to give coment in caller function which is bad callee function

-done watch: yes

13.-app: afroz smart acc

-observing app behaviour

-language: vb 6

-tool: die, x32dbg, hex workshop(remove static text)

-protection: -

-identify: string ref

-binary patch: method 1: jmp change byte

-done watch: yes

14.-app: acala dvd ripper

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-identify: string ref

-binary patch: method 1: jmp change byte

method 2: change eax inside callee function

-done watch: yes

15.-app: winmend folder

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: call stack

-binary patch: method 1: nop jmp change byte

-done watch: yes

16.-app: zc video converter

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-identify: call stack(xref method by string auto comment)

-binary patch: method 1: xor to or eax change instruction in function that triger bad branch

-some note: when use call stack to identify bad message, in top upper call stack(user party) there are many call in proram

so try find call<system function> no t call<user function> just one call<sytem function> it a big chance that

that is for bad message ex: call<Messagebox> if identified if not semm lik call<user32.xxxxx> this is for show

bad mesagebox.

-done watch: yes

17.-app: password remover

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: packer(upx),

bypass protection: pcaker: set breakpoint pushad till pop again then find oep and patch to disk with scyla plugin

-identify: call stack method(xref string auto comment)

-binary patch: method 1: nop change byte(not use)

method 2: xor to or change byte(by trace register in callee funtion that triger bad branch)

-done watch: yes

18.-app: tubedigger

-observing app behaviour

-language: c++

-tool: die, x32dbg, resource hacker(because there no POI can found in x32dbg)

-protection: themida[(packer)not detection by pe view tool], antidebug

bypass protection: use scyllahide plugin

-identify: constant memory(string ref from reource hacker)

-binary patch: method 1: nop change byte

-some note: 1.in pe view tool there no detect themida protector, to know it use schylahide plugin try one by one loda profile

in list(armadillo,vmprotect,themida,etc) when select themida it work program not detected debugger and still

run,while other debugger detected and program terminate earlier.

2.because it use protector many method dont work to identify point of interest

-by string refrence in messagebox to register -> doesnt work because when set breakpoint it dont hit each of them

-by string refrence with bad massagebox -> doesnt work because it not found when search in refrence.

-by memory map find pattern ->

-by constant memory -> work with load exe to resource hacker then find bad/point of interst string then copy it constant value

then paste it in search for-current module-constant-signed in x32dbg and found two, then set breakpoint

each of them when try again triger register vrsion it hit breakpoint, and assume one for bad message

other two bad message

-done watch: yes

19.-app: tag and rename

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: call stack(beacuse string doesnt work)

-binary patch: method 1:

-some note: in order to find correct message or sucessful branch it must have some jump instruction,

-if there is no jump in function that mean it wrong function,

-if just jmp message it mean also worng function,

-if also use call <register> mean wront function.

-if after call there is no logic jump(just jmp instruction) it strong indicate wrong function

-if same like previous points go to upper function

usually there is jump with conditional, xor, test . it bigger chance to assume it right function

after get right function from call stack try indicate bad function that corelate to bad branch in function

in simple word this function in bad branch, other branch must be good branch(05:07)

-done watch: yes

20.-app: inteligent shutdown

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: crc

bypass protection: see full video

-identify: call stack(beacuse string ref doesnt work)

-binary patch: method 1:

-some note: in die not detect it use protecor,after check in crypto there is modul crc mean app is protected, this protected

behaviour check integrity of file,after patch file program will not run because crc check

-done watch: yes

21.-app: flash demo

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: call stack method

-binary patch: method 1: nop change byte,jmp change byte

-some note: if in call stack there is one end jump biggest chance it wrong function, if it have two diferent jump biggest chance

it corect function

-done watch: yes

22.-app: http analyzer

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: string ref

-binary patch: method 1: jmp change byte

method 2: nop change byte

-done watch: yes

23.-app: sublime text

-observing app behaviour

-language: c++

-tool: die, x64dbg

-protection: -

-identify: string ref, attach

-binary patch: method 1: value change byte(crack "about" button)

method 2: intermodular call(crack "unregistered" string in main windows)

method 3: change xor to or instruction in callee function

-some note: in header windows "Unregistered" it use windows api fucntion(SetWindowText).

when attach program app most of in system module not main module usually in ntdll.dll, to switch module symbol->select

most top main module not system module double click on it

-done watch: yes

24.-app: pdf password remover

-observing app behaviour

-language: vb

-tool: die, x32dbg

-protection: -

-identify: attach, call stack

-binary patch: method 1:nop change byte

-some note: attach method same as previous video(sublime text). attach->symbol->double clik in main module not system modul

-done watch: yes

25.-app: easy work

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: -

-identify: trigering WaitMessage api(step in function api till back to main function with auto coment string identify)

-binary patch: method 1:mova al,1 and ret change instruction(that skip entire after it instruction)

-some note: attach method same as previous video(p23,24). attach->symbol->double clik in main module not system modul

WaitMessage api can see with right clikc->goto->expression->type WaitMessage

to exit from wait message box place function that call it with mov al,1, then ret because first instruction

need 2 byte and it use next byte, this next byte come to aunexpected instruction so change it with ret

-done watch: yes

26.-app: abacre restaurant point of sale

-observing app behaviour

-language: delphi

-tool: die, x32dbg, resource hacker(to remove trial and day left string)

-protection: -

-identify: string auto comment(from outer onion method), string ref

-binary patch: method 1: onion layer return method(WaitMessage same with preivious video p25) to remove nag screen

method 2: ret change byte[mov al,1; ret](to remove trial string)

-some note: onion layer method is coming out inner call layer by layer. by step out ret to out inner layer to higher method layer

use for wait event in messagebox to back to caller function.

see video for detail or in some note point 8

-done watch: yes

27.-app: head master seo

-observing app behaviour

-language:

-tool: die, x64dbg

-protection:

-identify: string ref

-binary patch: method 1: or rax,rax to or al,1 change byte

-some note: put breakpoint as many as program can show it, if to higher put breakpoint sometime program cant show messagebox

or unexpected behavior

sometime dont just focus on rax or other register, focus on cmp instruction that triger bad/good branch even cmp

is to rsp,ds,other.

-done watch: yes

28.-app:lmd extractor

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

29.-app: afroz pos

-observing app behaviour

-language: vb

-tool: die, x32dbg, vb decompiler, hex editor(to remove string manual)

-protection: -

-identify/point of interest: memory adress from vb decomp(string auto comment)

-binary patch: method 1: nop change byte

-some note: use vb decompiler to get point of interest(load form) copy it adress to x32dbg

-done watch: yes

30.-app: coper pos

-observing app behaviour

-language: c+

-tool: die, x32dbg

-protection: antidebug

-identify: string ref, attach

-binary patch: method 1: nop jmp change byter

-some note: because program have anti-debug in this chapter use attach program, after atach execution go to ntdll.dll(system module)

to go app module symbol->top upper app name double click->run(to syncronize with runing program)

same with p23,24

-done watch: yes

31.-app: dds

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: server check

-poi: step over stop at server check call

-binary patch: method 1: call server check change to nop change byte

-done watch: yes

32.-app: xpress2

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-poi: trace over stop at nag call[remove nag](because call stack method cant use), intermodullar call GetLocalTime

-binary patch: method 1: nop change byte

method 2: jmp change byte

-some note: when try call stack method user party just one and coment is ntdll.dll which it is system module not user module

so it cant use stack method.

to use intermodullar call right click->search for->current module->intermodular call

because there are 4 get local time, comment each breakpoint to trace it use. execute till return,then step over

then repeat until find test eax or cmp eax instruction. repeat step over and execute till return because it use

to know which caller fnction that call GetLocalTime api(like outer onion method).

after know which caller then compare app that still period time use and app that expired time, after get behavior

of app still period time than patch expired one to follow this active period app.

-done watch: yes

33.-app: ots

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-poi: nag call, string ref

-binary patch: method 1: jmp change byte

method 2: ret change byte

-some note: all entire function is nonsense to call nag, directly change prolog to ret instruction

-done watch: yes

34.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

35.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

36.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

37.-app: tuneskit

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-poi: attach, string ref

-binary patch: method 1: nop change byte

method 2: jmp change byte

-some note: when attach dont forget to back execution to user module, symbol->user_module_name(in party see user)->run to syncronize

when follow string ref it havenot logic function mean it wrong function(function that implement bad message) not implement

good and bad branch. repeat execute till return to caller that have logic branch

-done watch: yes

38.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

39.-app: winrar

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-identify: attach, intermodular GetcLocalTime api

-binary patch: method 1: nop change byte (in calle function after call GetLocalTime api)

-some note: dont forget to back execution to user module when attach

-done watch: yes

40.-app: mirrorOp

-observing app behaviour

-language: c++

-tool: die, x32dbg, http analyzer

-protection: -

-identify: string ref

-binary patch: method 1: nop change byte, server host block

-some note:to block server check run notepad to save url then go to C\Windows\System32\drivers\etc\host. this file check by os before

going to dns server. open file with notepad then add new line add: 127.0.0.1 url_name. this will loopback url_name to localhost

instead to url for ex:

url is https://www.krs<....>.com

in notepad just type 127.0.0.1 www.krs<...>.com -> with out http parameter,query, and so on

whenever program skip the block and go to the end it mean something wrong

-done watch: yes

41.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

42.-app: social phone extractor

-observing app behaviour

-language: .net

-tool: die

-protection:

-identify:

-binary patch: method 1:

-done watch: no

43.-app: internet download manager

-observing app behaviour

-language: c++

-tool: die, x32dbg, http analyzer

-protection: anti debug, server check

-poi: call stack, onion layer.

-binary patch: method 1:nop change byte, server host check block

-some note: bypass anti-debug use attach in debugger

dont forget when attach back the execution to user module, in symbol->user_module_name->click run to syncronize

because when out to parent function/execute til return, but program end in user32.dll which it is system module.

use run to user code icon, then after click ok in app then debugger automatically run till back to user module

onion layer use because when click ok, it in not correct funtion just function that implement bad branch. it can

detect by there no logic to bypass bad mesag.

-done watch: yes

44.-app: total comander

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: anti-tamper(anti analysis), fool reverser license not in help menu but in wincmd.key file,

crc / anti temper(force close after crack license)

-poi: strin ref,

onion layer (steping over return to go to parnt function),

lea data section (unread name register)

SetTimer os api (to remove crc/anti temper)

message breakpoint (to capture callbak funtion that triger force close)

-binary patch: method 1: jump change byte(bypass register)

method 2: adress lea change byte mem(data username),

inject data section(fix unreadable username)

method 3: jump change byte(remove anti-tamper)

method 4: jump change byte (bypass force close)

-some note: pe view cant detect anti-tamper

because anti-temper force close when click button it cant set breakpint, so use breakpoint message in debugger

handle->right click->refresh->search button string->right click->message breakpoint->chose buton type

->break on current windows only->use translate message

-done watch: yes

45.-app: encdata

-observing app behaviour

-language: c++ (unpack version)

-tool: die, x32dbg, api monitor

-protection: upx

-identify: popad and pushad (bypass upx method 1)

VirtualAlloc and VirtualProtect (bypass upx method 2)

string ref

onion layer (string auto comment, branch linear identifiaction)

config.data and keychain.dat (from api monitor app)

-binary patch: method 1: hooking (method 1 bypass upx seting bp in oep)

method 2 : jmp change byte

-some note: methods to unpack upx:

Method 1: push ebp method

put hw bpx on ebp wish is push ebp or pushad, then bpx is hit, trace to the jump to the oep and dump that memory.

Method 2: generic VirtualAlloc-VirtualProtect method

[2 step process to detect unpacking]

hook virtualalloc and virtualprotect.

if virtualallc eax returns the new base address which subsequently fills up with MZ.... you know it is unpacked. Then virtualprotect will be called to change to executable. Then you dump that memory

-done watch: yes

46.-app: wic reset key

-observing app behaviour

-language: c++

-tool: die, x32dbg, htttp analyzer(check is it use server when register)

-protection: -

-identify: call stack (string auto coment)

-binary patch: method 1: jump change byte

-some note: in call stack method bad function call before branch mean it function is not correct go to parent function.

-done watch: yes

47.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

48.-app: partition master

-observing app behaviour

-language: c++

-tool: die, x32dbg, process explorer (program use loader to excute main program)

-protection: -

-identify: string ref, attach, cxxThrowException (error cant run main.exe), verifyLicense function name in dll.

-binary patch: method 1: push 0 change byte (to enable button submit)

method 2: jump change byte (bypass loader)

method 3: nop change byte

-done watch: yes

49.-app: hexinator

-observing app behaviour

-language: py/c++

-tool: die, x64dbg

-protection: turbo activate (not detected by die, detected by entropy)

-identify: string ref, attach(bypass turbo activate),

-binary patch: method 1: nop change byte

-some note: to bypass turbo activate use attach.

-done watch: yes

50.-app: phantom pdf

-observing app behaviour

-language: c++

-tool: die, x32dbg, process explorer (detect derived process), api monitor (detect registration file program), hex editor(change static name)

-protection: -

-identify: CreaeFile api(api monitor program), regis file check, sring auto coment/function name

-binary patch: method 1: nop change byte

-done watch: yes

51.-app: bitlocker

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: -

-identify: string ref,attach

-binary patch: method 1: ds change byte / because patch branch doesnt work

-some note: if patch branch doesnt work try patch call before branch

-done watch: yes

52.-app: time table

-observing app behaviour: use watermark,cant export file

-language: c++

-tool: die, x32dbg, http analyzer, api monitor

-protection: server check

-identify: url monitor check(http analyzer), call stack method, deep call check, file read api check(api monitor app)

-binary patch: method 1: file host change, host redirect/block server check (c:\system32.....\host)

method 2: nop file deep patching call, reg file redirect

-done watch: yes

53.-app: html pad

-observing app behaviour

-language: delphi

-tool: die, x32dbg

-protection: server check, crc

-identify: string ref, nag function ref call, onion method

-binary patch: method 1: jump change byte

-some note: to find function refrence which call it, right click->find refrence to->selected address

see previous lesson how to bypass server check

to bypass crc because crc when click ok button it force close more worst is string encrypt cant find poi but

it is posible to track with pause show in nag screen. this video use deferent aproach with handle button when clik ok.

handle tab->right click(mouse in blank windows)->refresh->match string in button for ex button ok search "OK" string

->right click(in line string button)->message breakpoint->wm_lbuttonup(in message breakpoint nag)-> click break on

current window only(radio button)->use translate message. then click ok button execute will stop in system module

or loop message step till ret, after ret step till pass call<DispatchMessagew> then run to user code(click icon in debugger)

this video not use key license method, instead use expire trial method

-done watch: yes

54.-app: revo uninstaller

-observing app behaviour

-language: c++

-tool: die, x64dbg, resource hacker(register name injection)

-protection: -

-identify: deep call navigaion show,

-binary patch: method 1: jump change byte

method 2: change data segment string(change name in resource hacker)

method 3: parameter change byte(hiden window in api function parameter)

-some note: when nag is show execute automate pause, enter function callee deep till to function call system module or in function

when no navigation show but still pause there, it may correct function

-done watch: yes

55.-app: presentaion maker

-observing app behaviour

-language: c++

-tool: die, x32dbg, process explorer

-protection: -

-identify: string ref, getProfilePrivateString api, deep call check

-binary patch: method 1: ds:global variable change value

-some note: seem like keygen not simple patch like previous, use deep call check bcause patch branch doesnt work

-done watch: yes

56.-app: primo cache

-observing app behaviour

-language: c++

-tool: die, x64dbg

-protection: -

-identify: GetSystemAsFileTime api call, onion layer, string auto comment

-binary patch: method 1: nop change byte

-some note: these program have two way to crack, by license and by bypass expired.

before get corect api function there are so many "time" api call, to get correct function just type time in refrence

then set all to breakpoint then run program till finish, and see which breakpoint is hit. breakpoint which bever

get hit remove all of them

-done watch: yes

57.-app: gosync

-observing app behaviour

-language: c++

-tool: die, x32dbg, http analyzer, api monitor, hex editor

-protection: -

-identify: httpserver check request, Createfile api, string ref(file name string), string auto comment, call stack

-binary patch: method 1: server check blok/redirect host

method 2: or change byte in callee function

method 3: jump change byte

-done watch: yes

58.-app:dr account

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: antidebug,

-identify: call stack, string auto comment

-binary patch: method 1: jump change byte

method 2: register value change byte before ret to caller function

-some note: to bypass anti debug, very basic without plugn is debug->advanced->hide debugger

-done watch: yes

59.-app:

-observing app behaviour

-language: .net

-tool:

-protection:

-identify:

-binary patch: method 1:

-done watch: no

60.-app: ktab

-observing app behaviour

-language: c++

-tool: die, x32dbg

-protection: crc(anti-temper), call stack

-identify: string auto comment, nag pause execution

-binary patch: method 1: jmp change byte

-done watch: yes

61.-app: nitro pdf pro

-observing app behaviour

-language: c++

-tool: die, x64dbg, resource hacker

-protection: -

-identify: call stack, onion layer

-binary patch: method 1: jmp change byte

-done watch: yes

62.-app: crackme

-observing app behaviour

-language: c++

-tool: cheat engine, x32dbg,

-protection: upx

-identify: string ref

-binary patch: method 1: jmp change byte injection(cheat engine)

-done watch: yes

63.-app: crackme

-observing app behaviour

-language: c++

-tool: cheat engine, x32dbg

-protection: anti-debug, upx

-identify: string ref

-binary patch: method 1: jmp change byte injection(cheat engine)

-done watch: yes

64.-app: pasword remover

-observing app behaviour

-language: c++

-tool: die, x32dbg, cheat engine

-protection: upx

-identify: string ref

-binary patch: method 1: or change byte

-done watch: yes

65.-app: crackme

-observing app behaviour

-language: c++

-tool: die, x32dbg, cheat engine

-protection: upx

-identify: string ref, attach

-binary patch: method 1: cheat engine inject byte

-done watch: yes

66.-app: demo/test exe

-observing app behaviour

-language: c++

-tool: die, vmprotect,

-protection: vmprotect

-identify: string ref

-binary patch: method 1: jump change byte injection

-done watch: yes

67.-app: construct 2

-observing app behaviour

-language: -

-tool: die, cheat engine

-protection: vmprotect

-identify: memory adress read with cheat engine

-binary patch: method 1: inject process byte

-done watch: yes

-some note: this part is make keygen

68.-app: simple crackme

-observing app behaviour

-language: c++

-tool: die, enigma protector,

-protection: enigma

-identify: string ref

-binary patch: method 1: byte injection(trainer generate)

-done watch: yes

69.-app: simple demo crackme

-observing app behaviour

-language: -

-tool: die,themida orean,

-protection: themida

-identify: memory trace cheact engine

-binary patch: method 1: inject byte

-done watch: yes

some note: 1.if string refrence doesnt work, use reosurce hacker to get it, if still failed try next method is call stack

point 19

2.sometime change branch dont good choice because some app use verification in call function before check branch

this function before branch maybe use to make valid to other part ex: in point 19 change branch it suces to

bypass verification but in about string "unregistered" still show and it need to modify function return value before

branch to make string "unregistered" to suces license registered

3.use coment in bad branch/function that show bad message to make debugeer easier

4.aware when debug program sometime app use anti debug to prevent detection,

and sometime use fs:[] debug status to fool analyze ex: when in debugger it run normal but when run without debugger

it error, or sebaliknya

5.when attach program app most of in system module not main module usually in ntdll.dll, to switch module symbol->select

most top main module not system module in symbol for ex.

ex: user32.dll

kernel32.dll

ntdll.dll

sublime-text.exe <- although in 4th list this is top upper main module from user module

double click on it and program go to main module, and click run again

because in system module status is paused

in p23(sublime text 4th video),p24

6.principle of cracking: whenever branch to the last chunck it somthing bad

bypass lot number of black is bad in software cracking

7.in call stack aware that not just from party colum to see, from adress also important it in "from" colum

8.in onion layer method how to know which is correct function?

poin is how big is function, and many jump inside(not unconditional jump) at least one jump and if

auto comment string show up, it can see a lot interest string in that function.

after get bad message obviously we confuse which function call it, can see with click on prolog function

->right click ->find refrence to-> selected adress

9.one of interest trick is rather than check full fucntion for track return value, better change byte in prolog

with mov al,sucess_value

ret

this will make time effecincy for reverse engineer, than read nonsense function code

10.it take time trial and error to find correct place breakpoint, because sometime if dont correct breakpoint program

show unexpected behaviour p27

11. dont just focus on branch, if want growwh skill sometime need focus on cmp instrcution that triger bad/good bracnh

doesn mater it compare with rsp,ds,etc. p27

12. in softwware cracking bypass long module mean something wrong with branch, for ex: left branch lot check then go to

ret, while right branch it have a ton of module, mean that left branch bypass this ton of module and left branch indicate

that something wrong with it p27

13. know opcode is very important because when change byte to patch, must know which opcode is same or smaller than original

instruction, because if it big than original it overwrite the next instruction p 27(15:00)

14. analyze can be from top to down or from down to top, when one analyze so hard try swith other one. p28

15. if cant use string ref, call stack. try other method with trace over till show nag

16. there are many note from p27-32

17. if sure that all entire function is nonsense, just change to prolog to ret instruction p33

18. in call stack if found in comment is user_name module but in party is system mean it user module.

can see it in colum From and To p33

19. when string ref go to bad message that just implement bad function no logic branch, mean it just function that implement bad message

to know which is caller just execute till return rather than use xref that make time consuming and not productive, repeat execute

till return, till get function caller that have logic branch event it in 4th caller and so on p37

if in user module just ret function and other nonsense restart program when attach p39

20. to block server check see in p40

21. to bypass anti debug can use with plugin or use attach in debugger p43

22. if when attach it so hard to back to user module by use execute till return, always end in system module see p43 note

23. sometime dont user 127.0.0.1 as redirect server, try another ip adress because program know that trick

and set permision for that file because program always check adn change to correct url again, disable it

write mode p43

24. if to many poi like windows api, set all breakpoint even hundred when run if hit in it coment each of them till somepoint that

program done with it, every breakpoint dont forget give indeks coment to track it execute p44-51

25. to capture button that force close app, go to handle window in x32/64dbg and use set message breakpoint p44

26. in onion method if function few logic or no decision control and linear branch mean that not corret function, it just implementation function from branch p45

27. one of anti temper is cxxThrowException one of bypass it change branch that triger it p48

28. be carefull when patch is it exe of main program or dll p48

29. if patch branch doesnt work try patch call before branch p51

30. same with point 29 still not working patch deep as posible, this use when get exception when patch p52

31. in call stack althouh function seem like valid(many branch), keep think that if it in very top stack biggest chance

it function is not valid because too low in call stack, or if although function in more high call stack if it

very short function biggest chance it still in not correct function. p52

32. last chance exception explaint (p52,video 172 09:00-09:59)

33. when do patch program but get exception, then it must dig deeper to patch to make program naturaly take branch p52

34. if it hard to track function parameter/calling convention, use window below register window but manual read recomend to grow skill p3-55

35. to trace which call function is correct when come to keygen, not simple patch. generally trace eax and focus on it value p55

36. be aware of some global variable, it can also one of failed patch altough patch jump,return ax value. this global variable may effect

entire program different with local variable or register that very volatile. it posible that program use global variable as standar

to make decision through program. the data same through program even not distract p55

this posible as prevent analysis/anti analysis which fool reverser which just focus on branch and return value.

37. before get corect api function there are so many "time" api call, to get correct function just type time in refrence

then set all to breakpoint then run program till finish, and see which breakpoint is hit. breakpoint which bever

get hit remove all of them p56

38. to analyze which correct function usually after call there are lot/many branch p56

40. to change time in virtualbox kill VBoxService.exe process. this process get service from host to time p56

41. if just string auto comment left to use, this method require trial and error because string auto comment will show when execute the line

if go to bad branch string will appear if dont go to it string dont appear, so it must execute one by one to appear string auto comment

if get to not correct path must restart program again p56,57

42. when track by api monitor ex: monitor file api, to search it usually there are few interesting name [license,keydate,keyfile,keydata,serialdata,

serialdate,serialkey,serial file] but back to principal it depend PROGRAM TO READ FILE, it posible in list or it posible with unique or uknown name.

p57

43. in call stack method normally correct function not in very top stack user party it can be below it p57

44. in string auto comment, give own comment as posible because every step string auto comment is changes

keep eye on string auto comment in disassembly window,register windows, fastcall windows and stack memory windows p57

45. inject live memory instead patch p62

46. anti-debug will not detect cheat engine p63

47. trainer is like loader but better and powerful p67

48. if cant debug in debugger tool next option is cheat engine

49. in next option use cheat engine, but how if string is encrypted and cant find it? use memory scanning p69

method list: -string ref

-call stack method

-attach(p23,24,25)

-onion layer method(p26,43,45/new way,)[ret instruction,xref,find refrence to]

-attach(p30)

-execution automate pause when check validation(p31)

-intermodular api call(p32,39,44,45,48/upx bypass,56/trial time bypass)

-memory load lea instruction trace(p44)

-message breakpoint(p44) / refer to buton that triger force close app

-url monitor(p52)

-handle button when click force close[crc bypass](p53)

more in #CSL COURSE#

-deep call navigation show trace(p54)

-inject string with resource hacker(p54)

-keygen basic(p55 part 2)

-how to make keygen(p67)

protected list: -themida

how bypass p18

-upx

how bypass/unpack p17,45(2 method),62(full material),61,63,64,65(with cheat engine),

-vmProtect user/kernel level

how to bypass p66,67(with patch method,and make keygen GUI like general keygen found)

-enigma

hwo to bypass p68

-oreans themida

how to bypass p69

-turbo activate

how to bypass p49

-crc

how to bypass p20,44,53,60

-server check

how to block p31,40,43,52

-antidebug

how to bypass p30,43,58

-localtime api

how to bypass p32,39

-anti-tamper

how to bypass p44 force close temper,60

-loader to execute main program

how to bypass p48,49,50,61

tool list: -x64dbg,windbg,imunity debugger,vbdecompiler

-ida pro,cutter,binja,ghidra

-die,cff explorer, peid, exeinfo PE, pebear

-process explorer,api monitor,http analyzer,process hacker, resource hacker

-hex editor, hexworkshop

-cheat engine

my note: - sometime dont focus on in executable binary, program some of them use another file to valid license

- have to understand windows api to monitor program that use file somewhere to validate

- always aware is it excute in system module or user module

- in string auto comment, give own comment as posible because every step string auto comment is changes

keep eye on string auto comment in disassembly window,register windows, fastcall windows and stack memory windows

- before start to analysis check program behaviout with 3 essential tool:

-process explorer: check is program run another process

-http analyzer: check is program have server check

-api monitor: [optional] to check is program read/write file, or etc that realate to os api.

---

in course: done till 18

1)x64dbg tool

#G : view as graph

- : back to caller

s+s : go forward

pause: to stop animate/pause execution

f2: enable/disable break point not remove

right click->set breakpoint on all comand in refrence: to set hundred breakpoints in poi

ctrl+g: go to memory adress [right click->goto->expresion]

^

| : to show explain in below assembly windows, if explain tab is hiden try move cursor betwen dump assembly windows and dimp window

with that arrow not double up and down arrow just one to upper arrow

right click->find refrence to adress: shortcut to go to adress instead type it in command which make life dificult

can use to adress function,global variable,stack memory,etc

syntax: bp VirtualAlloc : set breakpoint for all virtualalloc function

1) advance dynamic analysis / debugger

#install first and observation app behavior

#use pe viewwer tools to see binary info like:programming language use, is it used protection, etc.

#try to find something unique as it can, like bad sring,error string,massagebox,etc. then TAKE A NOTE THIS UNIQUE IDENTIFIER

to mark when search breakpoint when open in debugger.

#open in debugger tool [here is x64dbg] nb: RUN AS ADMIMINISTRATOR THIS IS IMPORTANT

##setinng first for debugger:

###option->preference: in event disable all dialog except just two dialog entry breakpoint and attach breakpoint

in exception addd ignore exception from 0000-ffff

then save

###for plugin: load xanalyzer and scylla hide

#start seacrh breakpoint to analyse:

##by use string refrence : right click->search for->all user/current user modules->string refrence

search string that trigger errror or bad message, then set breakpoint all

of them if more than one, right click->togle breakpoint

#after get breakpoint to start analyze, then identify where the branch trigeer bad flow. try analyze each breakpoint if there is more than

one breakpoint, in breakpoint list that have mark right click->follow in disassembler

#in breakpoint use xanalyzer right click->xanalyzer->analyze module and right click->analyze->analyze module

then try scrolling to the top to get prologue function in prologue function click and right click->graph

#after in graph mode it more easier to identify, set breakpoint before jump to branch[JUMP] or in jump instruction then try

again with run icon in x64dbg tool, in app try again input that trigger bad message

after error message triger in app in debuger tool left bottom most there is word show it is PAUSED and in breakpoint line

there is litle bit black in red box

#after that in debuger tool left top->trace->trace record->word then step over to trace line, with trace active there are

color line that already trace or execute this make easier to trace flow of program

then trace until identify which brach act as bad or good message

#after get which branch triger bad or good message then identify which code that make branch to trigger bad message for example: before

branch there is cmp,test instruction mean this value make trigger branch, then identify which value that make this bracnh go to bad message

is 0,1 or other value, if not possible in cmp test instruction try before it that make cmp,test foe example like before cmp,test there is

function then in this function must to identify it return value that make cmp,test instruction use it as compare to triger branch

in case funtion to identify in graph mode->right click->sync with origin/rip, in function identify which return value that make triger bad branch

#after identify which value triger bad branch back to sequence mode right click->follow in disassembler->press space to change instrction

checklist keep size,fill with NOP's,asmjit. change instrction to good branch is from jnz to jmp,je to jmp, mov eax,1,etc. important is value/condition/instrction

that toward to good branch. then in space ui click x, try again in app and fill input to check is change code work that make app

to good message. if it work then patch

#patch is write memory program from ram to disk, in left top file->patch file->patch file(bottom right box)->name it with all file type->

if ok dialog appear,then x all dialog then stop debugger, close debugger. try run patch program is it success and check license

if it success to patch

a)some additional

#if there no string available use resource hacker to find it, in resource hacker->string->find->nput string to find

after get string copy it constant then in x64dbg insert it constant.

ex:string to find "Error message" opne resource hacker->clik icon serch->input string "Error mesage"

after show result 2888, "Error message". in x64dbg right click->search for->all/curent/user module

->constant->input in signed column with 28888, wait till x64dbg give result

ref:17,18

#if bypass work but still unregister show, must to surface patching check before branch deep into call find

what make it unsecesfull to show although bypass license work

#if want to mark where call function that open window app,

first enable mark trace->trace record->word

then start trace->animate over, this will stop at call window adress

ref: 06

#if rip in call before breakpoint right click->SYNC WITH RIP/ORIGIN

#if change memory with value must with decimal notation in x64dbg

#in graph mode if to large branch after trace over, use righclick->overview to see big picture of it that already trace

b)protection bypass

some protection like packer dont detected by pe view like die

#packer

#upx:-usually upx use two api LoadLibrary and Getprocess

-upx will extract original program from stub after call these two api function

-after extract upx will jump to OEP(orginial entry point)

-usually it patern is below jump instruction to OEP is many sequential zero byte and it istruction is mov eax,0

-after jump to OEP try to analysis is it OEP or some forward step to OEP

-after get OEP then patch it to disk from memory use scylla plugin

2)practice note

#if upx detect in DIE app, or in section there is upxo and upx1 section use manual unpacking recomend to growth skill

avoid automation unpack because it make lazy although it time productive

#method to search start breakpoint analys:

-with string refrence: if string refrence dont show branch it mean this function call by other function to triger error

right click->xref to see where function is call

3)analysis method

#by string refrence: right click->search for->...modules->strings refrece

#by constant memory that include string

in resource hacker icon find->inut string->after get string copy it constant value ex: 28888 "message"->copy 28888

in x64dbg right click->search for->....modules->constant->in signed column input constant ex: 28888

*this methode use if by string refence doesnt work

#call stack method

->first is triger error message in app, then click pause(left upper)

->then go to click in call stack icon,

->in call stack view find method usually in upper method with user party not system party,

but first check if it there is mark that where rip currently that make line call gray follow it if posible

->then right click->folow from

->after inside function right click->analysis->analyze module, then scroll upper to find prolog code to view in graph

->in graph view try to see where else does function to call right click->xref

->after get in caller function it show the function call, xref again but this caller function not first function

->repeat xref till get point where branch in function triger error

->if need to analysis call before jmp that triger error use step into function if not show in graph mode right click->syncronize with rip/origin

*this method use if by string or by constant memory include string doesnt work

4) patching method

#by change jump code/change branch

#by change value camulator register of register before jmp

#by change acumlator register in call that triger bypass

#by change instruction, for ex: use xor eax,eax change to or eax,eax

5)some tips

#dont over analysis just focus that what to patch,because app include obfuscator beside packer and encrypt. because obfuscator

include additional trash or useless code that give same result with simple code(control flatening obfuscator)

#dont analysis jump that make confuse this also may include branch obfuscator that add useless branch that give same result

with simple code

#focus on what thing that make bypass ex:rax triger error to bypass so trace rax dont look at other code just focus on

where rax value that make failed to bypass

#focus on what function do, don over analyst per line in function. focus on what it did and result

#if it not posible to bypass with change jmp instruction, change value in register that make jmp failed,

if change value register before jmp doesnt work, change value register inside call that make jmp failed

if change register value inside call doest work work go deep drill inside to 2ndfunction if doesnt work

god deeper to 3rd function,if same depper to 4th function and so on

#if use call stack method focus on where it appear ex: erro appear in mesage box, so go function that call messagebox in gui app

#use syn with origin to trace real time rip because sometime when after RET instruction there something hidden that make previous

change value handle by it. for ex: after change eax value in calee funtion when after RET instruction rip dont directly go to

caller function it jmp somewhere another module to change eax again(in csp p20)